

One Machine, Many Molds

One PU machine runs **many orders (products) at once**, each mounting **many molds**, within a **fixed number of slots**.

HYDRA MES — concept & implementation

1 • The Problem

The Setup

- One PU machine
- Fixed** physical slots
- Each slot holds **one mold**
- Many molds available in total

The Goal

Run **several orders in parallel** on the machine — each order = a different product — and let each product use **many molds at once**.

Wrong vs Right

- ✗ Model each mold as a separate machine
- ✓ One machine holding a **list of tools**, capped by **two capacity limits**

2 • The Idea

The Model

```
MACHINE (primary)
├ mold A   \
├ mold B   } secondary
└ mold N   / resources
```

Machine = primary resource
Molds = secondary resources
Slot = a capacity unit

Core Idea

The machine declares: I **hold a list of tools**.
Each order asks for: **N molds from its product pool**.
The system binds molds at logon and checks capacity.

Don't Confuse Two Shapes

N machines in parallel — one quantity split across separate machines.
1 machine, N mold slots — **this idea**. Molds mounted in slots on one machine.

3 • Real PU Case — 3 Tiers

3 Nested Tiers

```
MACHINE (fixed slots)
├ Order 1 = Product X → molds
├ Order 2 = Product Y → molds
└ Order 3 = Product Z → molds
```

Orders: 1..N (varies).
Molds/product: not fixed.
Slots: fixed.

Two Capacity Caps

Parallel orders →
Available capacity = N_orders × 1000
Parallel molds/slots →
Anonymous slot-pool Quantity = fixed slot count
Ceiling: max 20 OPs / machine.

Fixed Slots Enforced

HLS capacity-checks **the tools on an operation**, not just the machine.
Model slots as one **anonymous resource, Quantity = N**. Every OP consumes **anzahl** molds.
→ Σ molds across all orders ≤ N.

4 • How to Implement

1 • Define resources

- Machine = type **MNR**, marked **meta-resource**
- Each mold = type **WNR**
- Group molds into a **Family**

2 • Mold pool per product

Use a **Required resource**: one logical resource standing for a product's molds.
The system picks actual molds at logon.

3 • Slot-pool (fixed slots)

Create an **anonymous resource, Quantity = N** = physical slots on the machine.
This becomes the hard cap on total mounted molds.

4 • Cavities = partitioning

Each mold's **partitioning** = **number of cavities**.
Enable **Partitioning due to cavities** → output qty per cycle auto-calculated.

5 • Attach to the order

On the operation tool list add the **mold pool** (**anzahl** = molds) **and the slot-pool**.
Set **Log on with OP = Explicit** (operator scans mounted molds).

6 • Parallel + planning

- Logon of several OPs = Y** (≤20)
- Available capacity** = N_orders × 1000
- Plan **manual or auto** (**graptsbap**) — both honour both caps

5 • Runtime & Verify

Who Occupies Which Slot

Every mold gets its own occupancy record on the board:

- order
- lock
- maintenance window

Cavity Changes Mid-Run

Operator records opening / closing cavities at the terminal (reason: increase / reduction).
Logged to machine history; targets recalculate automatically.

Quality Per Slot

Quality data captured per cavity number.
Sample size = per-characteristic × number of cavities. Every part traceable to its mold.